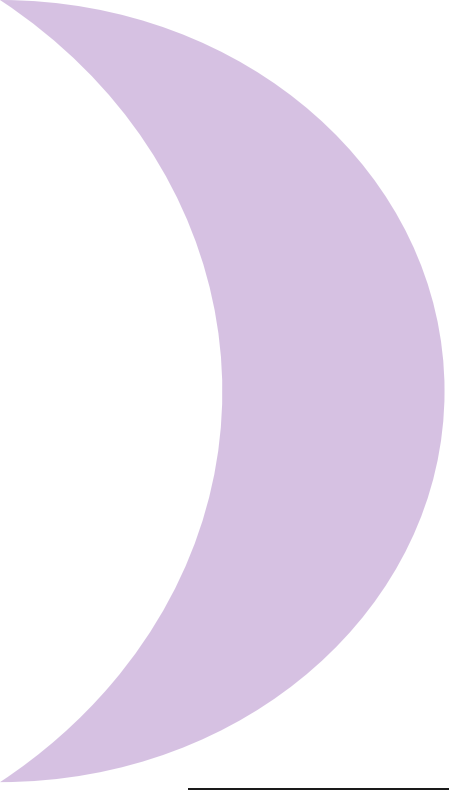




NUTRIHEALTH

Plataforma de Gestión Nutricional

Microservicios

Desarrollo Fullstack I

Integrantes: Antonia Maripan

Nicole Aliaga

Javier Parra

A. CONTENIDO DE LA EXPERIENCIA 2.

En el desarrollo de la segunda experiencia se vieron muchas tecnologías y herramientas las cuales son:

- **Arquitectura de Microservicios:** Lo implementamos fragmentando la aplicación en APIs independientes. Esto nos permite garantizar mayor tolerancia a fallo, debido a que si fallara alguna API el resto de APIs continuará activa, evitando afectar a todo el sistema, facilita las mejoras y un escalado individual por cada API del proyecto.
- **Spring Boot:** Usamos Spring Boot como el framework de JAVA base para el desarrollo del backend de cada API, este nos permite concentrarnos en la lógica del negocio y crear puntos de conexión del sistema de manera rápida y bien estructurada.
- **API Gateway:** El API Gateway la levantamos en el puerto 9090 (Evitamos posibles conflictos con 8080) . Esta actúa como nuestro único punto de entrada para las aplicaciones del cliente. a través de este llamamos al resto puertos, centralizando la solución.
- **WebClient:** Esto lo implementamos para resolver la comunicación interna y el intercambio de información entre las APIs. Con este logramos comunicarnos entre cada API en caso de necesitar información de otra.
- **Postman:** Lo utilizamos como nuestra herramienta principal en la experiencia 2 para validar y testear el comportamiento de cada endpoint de las APIs.
- **Gestión de Código (GitHub):** GitHub se utiliza como herramienta de control de versiones y para trabajar de forma colaborativa. El cual utilizamos el flujo basado en ramas (branch). Garantizando la estabilidad de nuestra rama main

B. DIAGRAMAS: MODELO DE BASE DE DATOS Y ARQUITECTURA

DIAGRAMA DE MODELO DE BASE DE DATOS



I. ¿EN QUÉ CONSISTE?

El servicio de autenticación es un componente centralizado responsable de verificar la identidad de los usuarios (pacientes y nutricionistas) que intentan acceder a la plataforma. Su función principal es recibir las credenciales (por ejemplo, correo y contraseña), validarlas mediante hashing seguro con `BCryptPasswordEncoder` y, si son correctas, firmar y emitir un JSON Web Token (JWT) de acceso con una validez temporal.

II. AUTENTICACIÓN POR TOKEN (JWT)

La autenticación por token se implementa comúnmente utilizando JSON Web Tokens (JWT). El flujo de autenticación es el siguiente:

1. El cliente envía sus credenciales al auth-service (endpoint de login).
2. El servicio valida las credenciales y genera un JWT firmado con una clave secreta. Este token contiene información del usuario como su ID, rol y tiempo de expiración.
3. El servidor devuelve el JWT al cliente.
4. Para cada petición subsecuente a recursos protegidos, el cliente incluye el JWT en el header HTTP Authorization (con el formato Bearer <token>).
5. El API Gateway o los microservicios verifican la firma y validez del token antes de permitir el acceso a la ruta solicitada.

En el proyecto, se implementa utilizando ****JWT****:

1. El usuario envía sus credenciales al endpoint de login (POST /auth/login)
2. El servicio valida las credenciales y genera un JWT firmado con una clave secreta JWT_SECRET. El token contiene metadatos como el correo electrónico del usuario, sus roles y la fecha de expiración.
3. El JWT es devuelto al cliente dentro del objeto AuthResponse.
4. El cliente almacena el token y lo adjunta en la cabecera Authorization: Bearer <token> de cada petición HTTP subsiguiente.
5. El API Gateway intercepta todas las peticiones con su filtro global AuthenticationFilter, extrae el token y valida su firma y expiración a través del servicio de JWT JwtService.validarToken. Si es válido, permite que la petición continúe al microservicio destino, de lo contrario retorna a un estado 401 Unauthorized.

III. SPRING SECURITY, PROTECCIÓN DE RUTAS Y CORS

SPRING SECURITY

Se configura en el microservicio auth-service para deshabilitar CSRF (*Cross-Site Request Forgery*, o falsificación de petición en sitios cruzados), al ser una API sin estado, definir políticas de sesión sin estado (`STATELESS`) y dar acceso público solo a los endpoints bajo `/auth/\*\*` (registro/login) y a la documentación de OpenAPI, protegiendo todos los demás endpoints.

PROTECCIÓN DE RUTAS

En el API Gateway, el filtro `AuthenticationFilter` tiene una lista de prefijos de rutas públicas:

```
private boolean isPublicPath(String path) {
    List<String> publicPrefixes = List.of(
        "/auth/",
        "/swagger-ui",
        "/v3/api-docs",
        "/api/v1/nutricionistas/v3/api-docs",
        "/api/v1/pacientes/v3/api-docs",
        "/api/v1/recetas/v3/api-docs",
        "/api/v1/alimentos/v3/api-docs",
        "/api/v1/minutas/v3/api-docs",
        "/api/v1/antecedentes/v3/api-docs",
        "/api/v1/rutinas/v3/api-docs",
        "/api/v1/citas/v3/api-docs",
        "/api/v1/metast/v3/api-docs");
    return publicPrefixes.stream().anyMatch(path::startsWith);
}
```

Toda ruta que no coincida con esta lista es bloqueada si no posee un JWT válido.

Cada API tiene su propio filtro dependiendo del tipo de operación el filtro de security verifica si el usuario autenticado posee al menos uno de los roles permitidos en el hasanyrole.

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http, JwtAuthenticationFilter jwtAuthenticationFilter) throws Exception {
    return http
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/swagger-ui.html", "/swagger-ui/**", "/v3/api-docs/**", "/api/v1/pacientes/v3/api-docs", "/error")
            .requestMatchers(HttpMethod.GET, "**").hasAnyRole("ADMINISTRADOR", "ADMIN", "NUTRICIONISTA", "PACIENTE", "INTERNAL")
            .requestMatchers(HttpMethod.POST, "**").hasAnyRole("ADMINISTRADOR", "ADMIN", "NUTRICIONISTA", "INTERNAL")
            .requestMatchers(HttpMethod.PUT, "**").hasAnyRole("ADMINISTRADOR", "ADMIN", "NUTRICIONISTA", "INTERNAL")
            .requestMatchers(HttpMethod.DELETE, "**").hasAnyRole("ADMINISTRADOR", "ADMIN", "INTERNAL")
            .anyRequest().authenticated())
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

CORS (CROSS-ORIGIN RESOURCE SHARING)

Es un mecanismo de seguridad del navegador que restringe las peticiones HTTP de origen cruzado. Configurado a nivel global en el API Gateway mediante la anotación CrossOrigin permitiendo los orígenes del frontend (ej. `http://localhost:9090` u otros puertos en desarrollo) y

los verbos HTTP `GET, POST, PUT, DELETE, OPTIONS` para evitar el bloqueo del navegador al realizar peticiones de dominio cruzado.

```
# CONFIGURACIÓN GLOBAL DE CORS (Evita el Failed to Fetch)
spring.cloud.gateway.globalcors.cors-configurations.[/**].allowedOrigins=http://localhost:9090
spring.cloud.gateway.globalcors.cors-configurations.[/**].allowedMethods=GET,POST,PUT,DELETE,OPTIONS
spring.cloud.gateway.globalcors.cors-configurations.[/**].allowedHeaders=*
```

D. IMPLEMENTACIÓN POR CAPAS

La implementación por capas en Spring Boot divide la aplicación en diferentes responsabilidades bien definidas:

- **DTO:**
 - Capa de transferencia de datos. Son objetos simples que definen qué datos viajan entre el cliente y el servidor. Sirven para no exponer la estructura real de la base de datos.

```
1 package com.recetas.api_recetas.dto;
2
3 import lombok.Data;
4
5 @Data
6 public class AlimentosDto {
7     private Long id_alimento;
8     private String nombre;
9     private Double proteinaG;
10    private Double grasaG;
11    private Double carbohidratoG;
12    private Double calorias;
13
14 }
```

```
1 package com.recetas.api_recetas.dto;
2
3 import lombok.Data;
4
5 @Data
6 public class RecetaIngredienteDto {
7     private Long id_ingredientes;
8     private Double cantidadGramos;
9
10    private AlimentosDto alimento;
11 }
```

- **Model:**

- Representa el modelo de dominio y la estructura de los datos en la aplicación. Estas se mapean directamente con las tablas de la base de datos mediante anotaciones.

```
16 import lombok.NoArgsConstructor;
17
18 @Entity
19 @AllArgsConstructor
20 @NoArgsConstructor
21 @Data
22 @Table(name = "receta")
23 public class Receta {
24
25     @Id
26     @GeneratedValue(strategy = GenerationType.IDENTITY)
27     private Long id_receta;
28
29     @Column(nullable = false)
30     private Long id_paciente;
31
32     @Column(nullable = false)
33     private String nombre_plato;
34
35     @Column(nullable = false)
36     private String preparacion;
37
38     @Column(nullable = false)
39     private String estado;
40
41     @Column(nullable = true)
42     private String anotaciones;
43
44     @OneToMany(mappedBy = "receta", cascade = CascadeType.ALL)
45     @JsonManagedReference
46     private List<RecetaIngredientes> ingredientes;
47 }
```

```
16 @Entity
17 @AllArgsConstructor
18 @NoArgsConstructor
19 @Data
20 @Table(name = "receta_ingredientes")
21 public class RecetaIngredientes {
22
23     @Id
24     @GeneratedValue(strategy = GenerationType.IDENTITY)
25     private Long id_ingredientes;
26
27     @Column(nullable = false)
28     private Long id_alimento;
29
30     @Column(nullable = false)
31     private Double cantidadGramos;
32
33     @ManyToOne
34     @JoinColumn(name = "id_receta")
35     @JsonBackReference
36     private Receta receta;
37
38 }
```

- Estas clases representan las entidades de la base de datos mediante JPA. Las anotaciones `@Entity` y `@Table` mapean las clases a tablas específicas. Se definen las claves primarias autoincrementables con `@Id` y `@GeneratedValue`.
- Se gestionan las relaciones entre tablas: `RecetaIngredientes` usa `@ManyToOne` para vincularse a una receta, mientras que `Receta` usa `@OneToMany` para listar sus ingredientes. Las anotaciones `@JsonManagedReference` y `@JsonBackReference` son cruciales aquí para evitar bucles infinitos al momento de serializar las respuestas en formato JSON.

- **Controller:**

- Maneja las peticiones HTTP, extrae parámetros y delega el trabajo al servicio. Devuelve las respuestas HTTP.

```

21 @RestController
22 @RequestMapping("api/v1/recetas")
23 public class RecetaController {
24
25     @Autowired    Convert @Autowired field into Constructor Parameter
26     private RecetaService recetaService;
27
28     @GetMapping
29     public ResponseEntity <List<Receta>> listarReceta(){
30         List<Receta> recetas = recetaService.mostrarRecetas();
31         if(recetas.isEmpty()){
32             return ResponseEntity.noContent().build();
33         }
34         return ResponseEntity.ok(recetas);
35     }
36
37     @PostMapping
38     public ResponseEntity<Receta> guardarReceta(@RequestBody Receta unaReceta){
39         Receta recetaNueva = recetaService.crearReceta(unaReceta);
40         return ResponseEntity.status(HttpStatus.CREATED).body(recetaNueva);
41     }
42
43     @GetMapping("/{id}")
44     public ResponseEntity<Receta> buscarReceta(@PathVariable Long id){
45         try{
46             Receta receta = recetaService.buscarId(id);
47             return ResponseEntity.ok(receta);
48         }catch (Exception e){
49             return ResponseEntity.notFound().build();
50         }
51     }
52
53     @PutMapping("/{id}")
54     public ResponseEntity<Receta> modificarReceta(@PathVariable Long id, @RequestBody Recet
55         try {
56             Receta rec = recetaService.buscarId(id);
57             rec.setId_receta(id);
58             rec.setNombre_plato(unaReceta.getNombre_plato());
59             rec.setPreparacion(unaReceta.getPreparacion());
60             rec.setEstado(unaReceta.getEstado());
61             rec.setAnotaciones(unaReceta.getAnotaciones());
62
63             if (unaReceta.getIngredientes() != null) {
64                 for (RecetaIngredientes ing : unaReceta.getIngredientes()) {
65                     ing.setReceta(rec);
66                 }
67             }
68             rec.setIngredientes(unaReceta.getIngredientes());
69
70             Receta recetaGuardada = recetaService.crearReceta(rec);
71
72             return ResponseEntity.ok(recetaGuardada);
73         } catch (Exception e) {
74             return ResponseEntity.notFound().build();
75         }
76     }
77
78     @DeleteMapping("/{id}")
79     public String eliminarReceta(@PathVariable Long id){
80         try{
81             recetaService.eliminarReceta(id);
82             return "Receta eliminada";
83         }catch (Exception e){
84             return "La receta no existe";
85         }
86     }
87 }

```

El controlador, marcado con `@RestController` y `@RequestMapping`, expone los *endpoints* de la API para la gestión de recetas. Define los métodos HTTP que el cliente puede consumir

- **Service:**

- Contiene la lógica de negocio. Realiza validaciones y orquesta llamadas a los repositorios.

```
1 package com.recetas.api_recetas.service;
2
3 import java.util.List;
4 //import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.stereotype.Service;
6 import com.recetas.api_recetas.model.Receta;
7 import com.recetas.api_recetas.repository.RecetaRepository;
8
9 import jakarta.transaction.Transactional;
10
11 @Service
12 @Transactional
13 public class RecetaService {
14
15     private final RecetaRepository recetaRepository;
16
17     public RecetaService(RecetaRepository recetaRepository) {
18         this.recetaRepository = recetaRepository;
19     }
20
21     public List<Receta> mostrarRecetas(){
22         return recetaRepository.findAll();
23     }
24
25     public Receta buscarId(Long id){
26         return recetaRepository.findById(id).get();
27     }
28
29     public Receta crearReceta(Receta receta){
30         return recetaRepository.save(receta);
31     }
32
33     public void eliminarReceta(Long id){
34         recetaRepository.deleteById(id);
35     }
36 }
37
```

- **Repository:**

- Capa de acceso a datos, se comunica con la base de datos (generalmente mediante Spring Data JPA).
- Esta interfaz extiende JpaRepository, recibiendo como parámetros la entidad (Receta) y el tipo de dato de su clave primaria (Long). Al heredar de esta interfaz de Spring Data, el microservicio obtiene automáticamente todos los métodos CRUD (crear, leer, actualizar, borrar).

```

1  package com.recetas.api_recetas.repository;
2
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import org.springframework.stereotype.Repository;
5
6  import com.recetas.api_recetas.model.Receta;
7
8  @Repository      Unnecessary @Repository
9  public interface RecetaRepository extends JpaRepository<Receta, Long>{
10
11  }
12

```

I. EVIDENCIA DE CAPTURA DE PANTALLA POR VERBO HTTP

ANTES DE PODER REALIZAR EL TEST EN POSTMAN SE NECESITA GENERAR EL TOKEN E INGRESAR.

POST — CREAR USUARIO CON TOKEN

The screenshot shows the Postman interface for a POST request to `http://localhost:8090/auth/register`. The request body is a JSON object:

```

{
  "nombreUsuario": "Josefa Ramirez",
  "contrasena": "MiPass123",
  "correo": "joramirez@gmail.com",
  "roles": ["ADMINISTRADOR"]
}

```

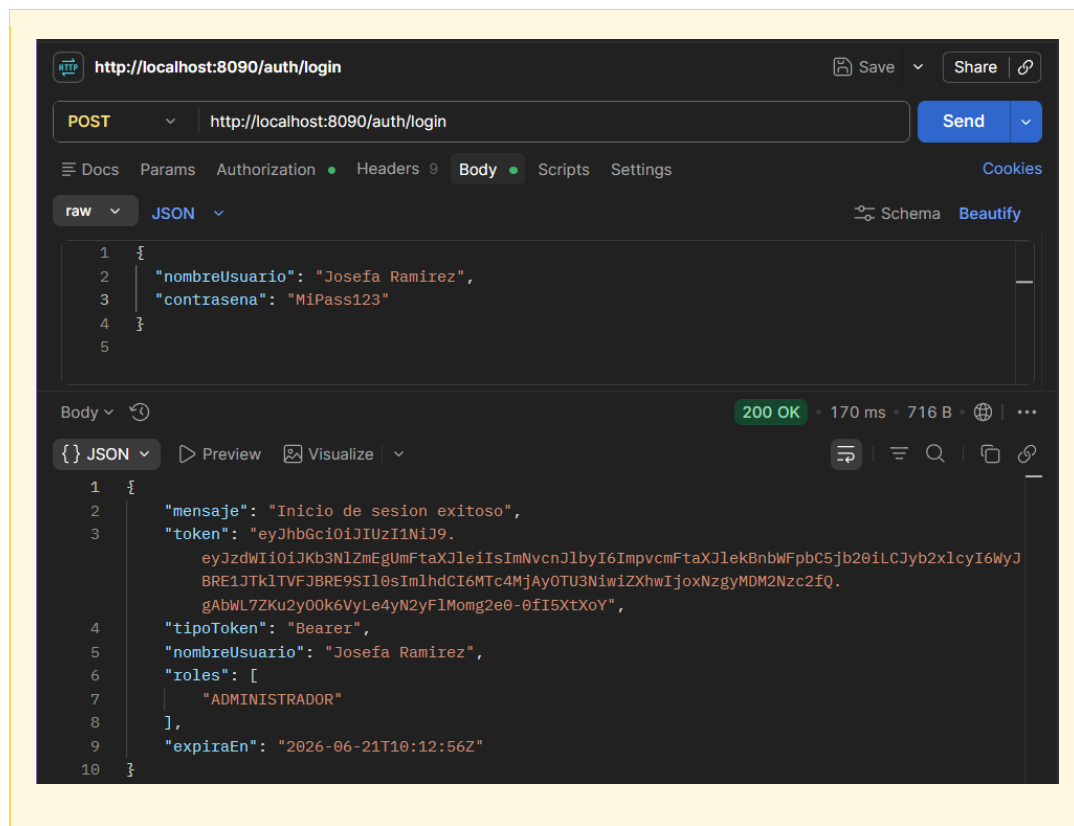
The response is a **201 Created** status with a response time of 196 ms and a body size of 729 B. The response body is a JSON object:

```

{
  "mensaje": "Usuario registrado correctamente",
}

```

POST — LOGIN CON TOKEN



POST — CREAR RECURSO

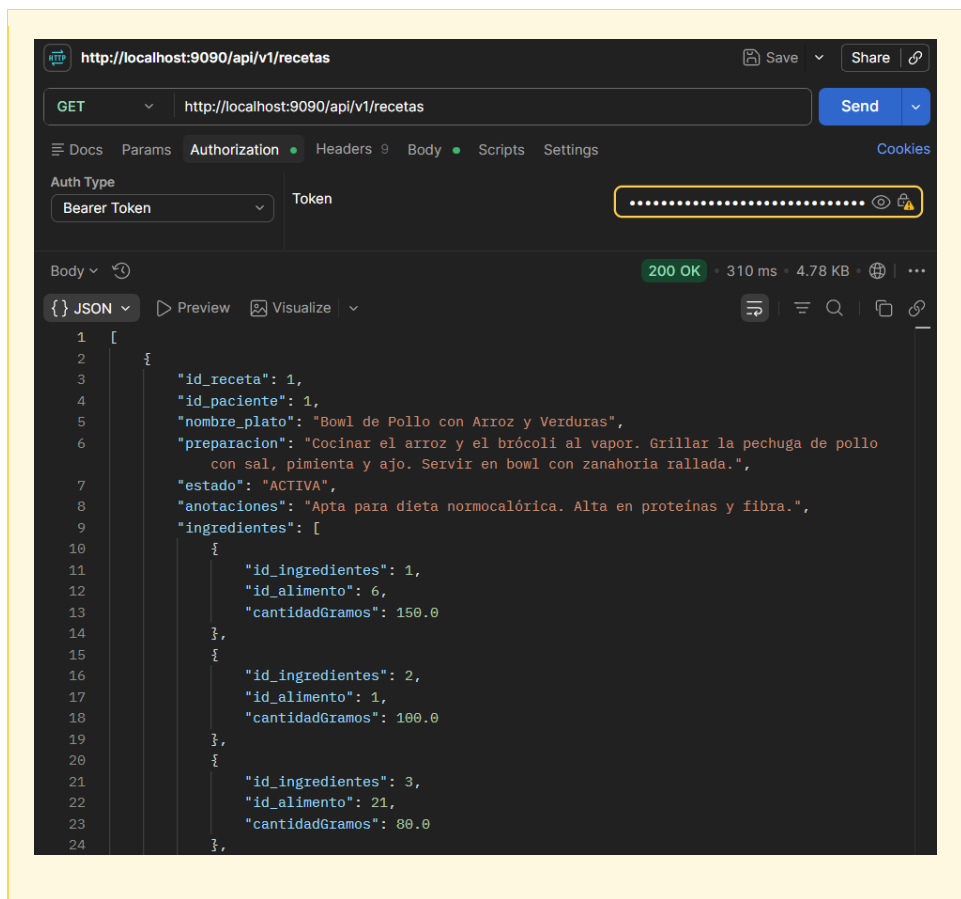
Se le asigna una receta a un paciente, junto a los id de los alimentos que representan los ingredientes que necesita esa receta. Devuelve un 201 cuando se crea.

```
1 {
2   "id_paciente": 5,
3   "nombre_plato": "Ensalada Vegetariana de Lentejas",
4   "preparacion": "Cocinar lentejas, mezclar con tomate, pepino, zanahoria y palta. Aliñar con aceite de oliva y limón.",
5   "estado": "ACTIVA",
6   "anotaciones": "Receta vegetariana rica en proteína vegetal.",
7   "ingredientes": [
8     { "id_alimento": 4, "cantidadGramos": 150.0 },
9     { "id_alimento": 24, "cantidadGramos": 80.0 },
10    { "id_alimento": 25, "cantidadGramos": 60.0 },
11    { "id_alimento": 23, "cantidadGramos": 50.0 },
12    { "id_alimento": 27, "cantidadGramos": 100.0 },
13    { "id_alimento": 26, "cantidadGramos": 10.0 }
14  ]
15 }
```

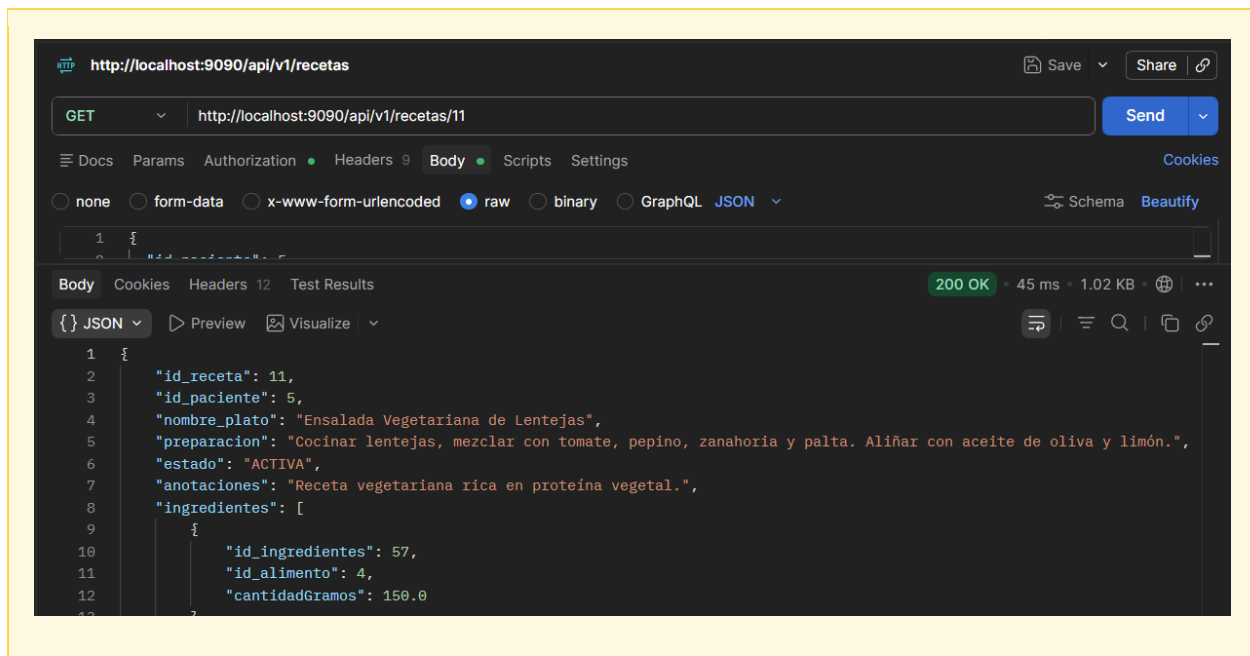
```
1 {
2   "id_receta": 11,
3   "id_paciente": 5,
4   "nombre_plato": "Ensalada Vegetariana de Lentejas",
5   "preparacion": "Cocinar lentejas, mezclar con tomate, pepino, zanahoria y palta. Aliñar con aceite de oliva y limón.",
6   "estado": "ACTIVA",
7   "anotaciones": "Receta vegetariana rica en proteína vegetal.",
8   "ingredientes": [
9     {
10      "id_ingredientes": 57,
11      "id_alimento": 4,
12      "cantidadGramos": 150.0
13    }
14  ]
15 }
```

GET — OBTENER TODOS

Al hacer un get de todas las recetas, se pueden ver los ingredientes asignados a cada receta.

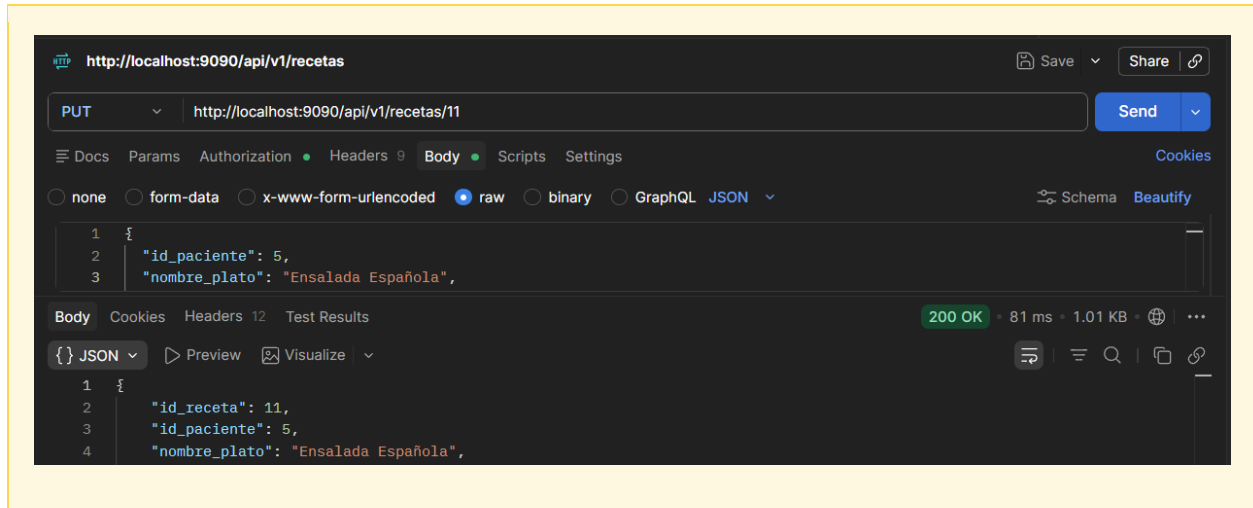


GET POR ID — OBTENER ESPECÍFICO



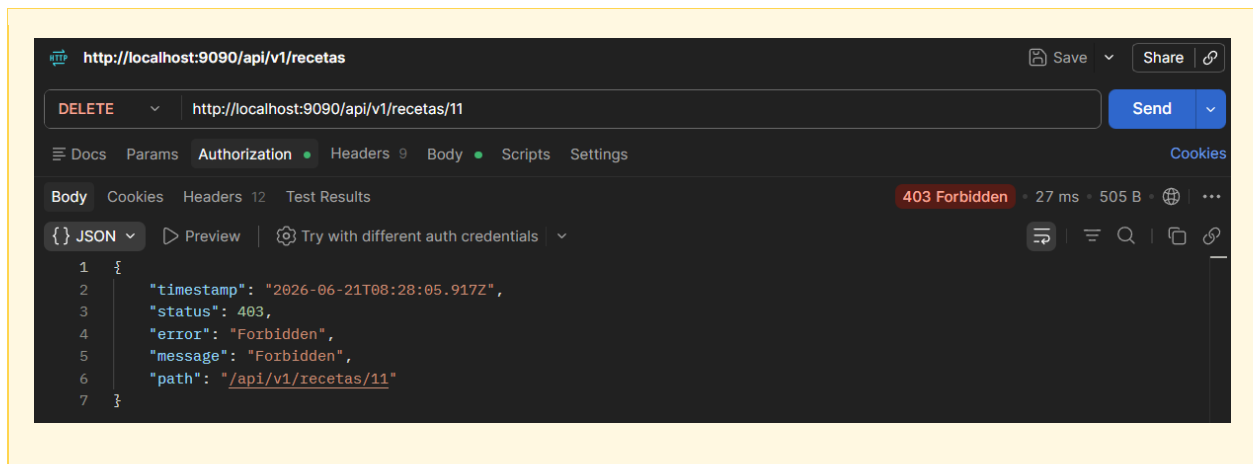
PUT POR ID — ACTUALIZAR RECURSO

Se actualiza nombre de receta.

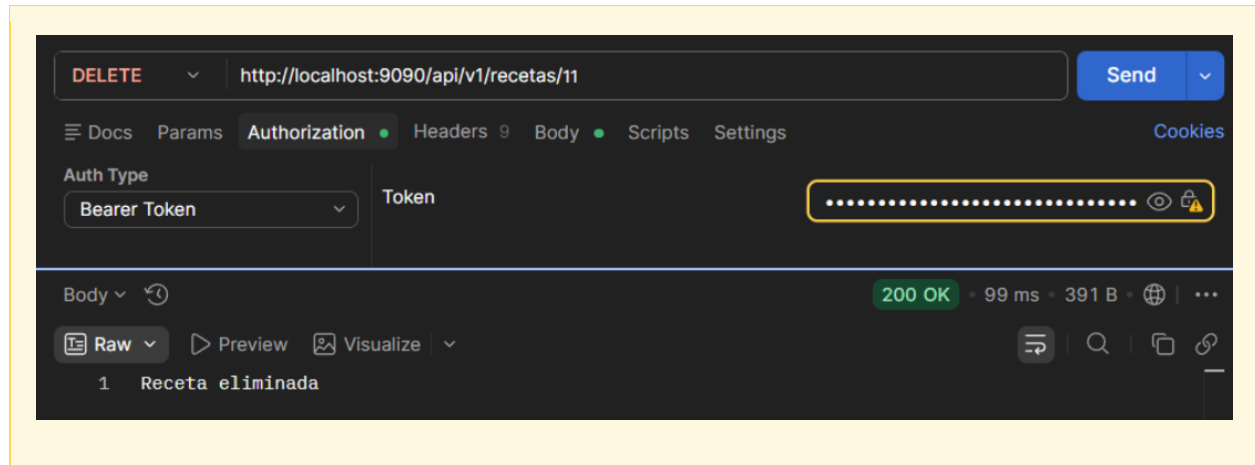


DELETE POR ID — ELIMINAR RECURSO

Solo rol administrador puede borrar, si se intenta borrar desde otro rol, como por ejemplo paciente, aparece un 403 porque ese rol no tiene permisos para eliminar nada.

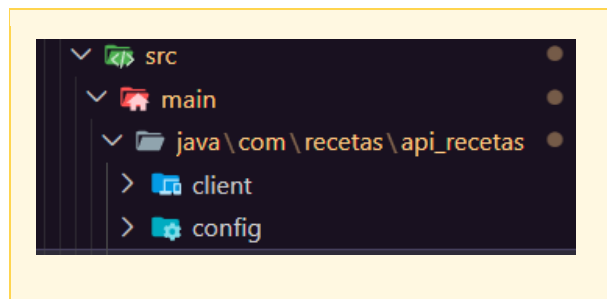


Desde admin:



II. COMUNICACIÓN ENTRE MICROSERVICIOS

Se añaden nuevos packages al main para implementar WebClient



```

ExternalServiceClient.java X
backend > api-recetas > src > main > java > com > recetas > api_recetas > client > ExternalServiceClient.java > Lang
1 package com.recetas.api_recetas.client;
2
3 import org.springframework.stereotype.Service;
4 import org.springframework.web.reactive.function.client.WebClient;
5
6 import com.recetas.api_recetas.dto.AlimentosDto;
7
8 @Service
9 public class ExternalServiceClient {
10
11     private final WebClient webClient;
12
13     public ExternalServiceClient(WebClient webClient) {
14         this.webClient = webClient;
15     }
16
17     public AlimentosDto obtenerAlimento(Long id_alimento) {
18         try{
19             return webClient
20                 .get()
21                 .uri("http://localhost:8083/api/v1/alimentos/" + id_alimento)
22                 .retrieve()
23                 .bodyToMono(AlimentosDto.class)
24                 .block();
25         } catch (Exception e){
26             return null;
27         }
28     }
29
30 }

```

Para habilitar la comunicación interna, se implementó un cliente HTTP reactivo usando WebClient. La clase ExternalServiceClient realiza una petición HTTP GET de forma síncrona (usando .block()) al microservicio de alimento. Pasa el ID del alimento en la URL, recupera el JSON resultante y lo mapea automáticamente al objeto AlimentosDto. Esto permite que el servicio de recetas obtenga información de los alimentos sin acceder directamente a la base de datos del otro microservicio.

```

WebConfig.java X
backend > api-recetas > src > main > java > com > recetas > api_recetas > config > WebConfig.java
1 package com.recetas.api_recetas.config;
2
3 import org.springframework.beans.factory.annotation.Value;
4 import org.springframework.context.annotation.Bean;
5 import org.springframework.context.annotation.Configuration;
6 import org.springframework.web.reactive.function.client.WebClient;
7
8 @Configuration
9 public class WebConfig {
10
11     @Value("${security.internal.secret}")
12     private String internalSecret;
13
14     @Bean
15     public WebClient webClient(){
16         return WebClient.builder()
17             .defaultHeader("X-Internal-Request", internalSecret)
18             .build();
19     }
20 }
21

```

En la clase WebConfig declarada con @Configuration, se construye y expone el Bean de WebClient. Aquí se configura una capa de seguridad para la comunicación interna: se añade un encabezado HTTP por defecto (X-Internal-Request) que contiene un secreto (internalSecret) inyectado desde las

variables de entorno (@Value). Esto asegura que el microservicio receptor sepa que la petición proviene de un servicio interno confiable.

E. SWAGGER

I. ¿QUÉ ES Y EN QUÉ CONSISTE?

Swagger (ahora parte de la iniciativa OpenAPI) es un conjunto de herramientas para diseñar, construir, documentar y consumir APIs RESTful. Consiste en una interfaz gráfica (Swagger UI) autogenerada a partir del código que permite visualizar y probar de forma interactiva todos los endpoints disponibles en la aplicación, sus parámetros esperados, y los modelos de respuesta.

II. ¿CÓMO SE IMPLEMENTA EN UN PROYECTO DE MICROSERVICIOS CON SPRING BOOT?

En Spring Boot, se implementa fácilmente agregando la dependencia en el archivo pom.xml.

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.8.13</version>
</dependency>
```

Una vez agregada, Springdoc escanea automáticamente los controladores (@RestController) y sus métodos (@GetMapping, @PostMapping, etc.) para generar la documentación OpenAPI.

Se puede acceder a la interfaz gráfica accediendo a la URL /swagger-ui.html o /swagger-ui/index.html de cada microservicio.

III. EVIDENCIA CON CAPTURAS DE PANTALLA

Se ingresa al siguiente link para poder crear token e ingresar a swagger

- <http://localhost:8090/swagger-ui/index.html#>

I. ¿EN QUÉ CONSISTE? ¿POR QUÉ ES IMPORTANTE TESTEAR?

El testing consiste en someter el software a pruebas para verificar que funciona según lo esperado y detectar errores. Es crucial por las siguientes razones:

- Asegura la calidad y fiabilidad del software.
- Previene regresiones (que nuevos cambios rompan funcionalidades existentes).
- Facilita el mantenimiento y la refactorización del código de manera segura.
- Sirve como documentación viva de qué debe hacer el código.

II. PRUEBAS UNITARIAS — PATRÓN AAA

Las pruebas unitarias evalúan el comportamiento de unidades individuales de código (generalmente métodos o clases) de forma aislada. El patrón AAA estructura la prueba en tres pasos:

ARRANGE (PREPARAR)

Se inicializan los objetos, se configuran los mocks (datos simulados) y se preparan las precondiciones necesarias para la ejecución de la prueba.

ACT (ACTUAR)

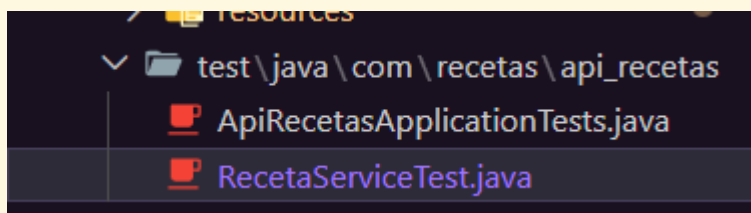
Se invoca el método o funcionalidad que se está probando con los datos preparados en la fase anterior.

ASSERT (AFIRMAR)

Se verifican los resultados obtenidos contra los resultados esperados (por ejemplo, validando retornos o comprobando que se llamaron ciertos métodos de dependencias).

III. EVIDENCIA POR MÉTODO IMPLEMENTADO EN CADA CAPA

En la carpeta de test, que genera automáticamente springboot, implementar la clase RecetaServiceTest para hacer el testing del service creando un objeto simulado con mockito.



```

21
22 @ExtendWith(MockitoExtension.class)
23 public class RecetaServiceTest {
24
25     @Mock
26     private RecetaRepository recetaRepository;
27
28     @InjectMocks
29     private RecetaService recetaService;
30
31     private Receta ejemploReceta;
32
33     @BeforeEach
34     void setUp() {
35         ejemploReceta = new Receta();
36         ejemploReceta.setId_receta(1L);
37         ejemploReceta.setId_paciente(1L);
38         ejemploReceta.setNombre_plato(nombre_plato: "Ensalada Griega");
39         ejemploReceta.setPreparacion(preparacion: "Mezclar ingredientes frescos");
40         ejemploReceta.setEstado(estado: "Activa");
41         ejemploReceta.setAnotaciones(anoaciones: "Sin sal");
42     }
43
44     @Test
45     @DisplayName("Mostrar todas las recetas")
46     void mostrarRecetas() {
47         when(recetaRepository.findAll()).thenReturn(List.of(ejemploReceta));
48
49         List<Receta> resultado = recetaService.mostrarRecetas();
50
51         assertNotNull(resultado);
52         assertEquals(1, resultado.size());
53         assertEquals("Ensalada Griega", resultado.get(index: 0).getNombre_plato());
54         verify(recetaRepository, times(1)).findAll();
55     }

```

Utilizando JUnit y Mockito, se crea la clase de pruebas para aislar la capa de servicio. Se usa `@Mock` para crear una simulación (mock) del `RecetaRepository` y `@InjectMocks` para inyectar este repositorio simulado dentro del `RecetaService`. El método anotado con `@BeforeEach` garantiza que antes de ejecutar cada prueba, se instancie un objeto `Receta` con datos de prueba estandarizados ("Ensalada Griega"), manteniendo un entorno limpio y predecible.

```

57
58 @Test
59 @DisplayName("Buscar receta por id")
60 void buscarId() {
61     when(recetaRepository.findById(id: 1L)).thenReturn(Optional.of(ejemploReceta));
62
63     Receta resultado = recetaService.buscarId(id: 1L);
64
65     assertNotNull(resultado);
66     assertEquals(1L, resultado.getId_receta());
67     assertEquals("Ensalada Griega", resultado.getNombre_plato());
68     verify(recetaRepository, times(1)).findById(id: 1L);
69 }
70
71 @Test
72 @DisplayName("Crear una receta")
73 void crearReceta() {
74     when(recetaRepository.save(ejemploReceta)).thenReturn(ejemploReceta);
75
76     Receta resultado = recetaService.crearReceta(ejemploReceta);
77
78     assertNotNull(resultado);
79     assertEquals("Ensalada Griega", resultado.getNombre_plato());
80     verify(recetaRepository, times(1)).save(ejemploReceta);
81 }

```

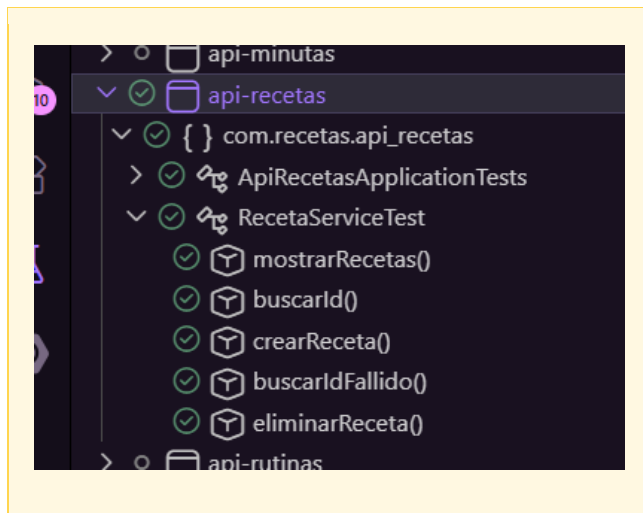
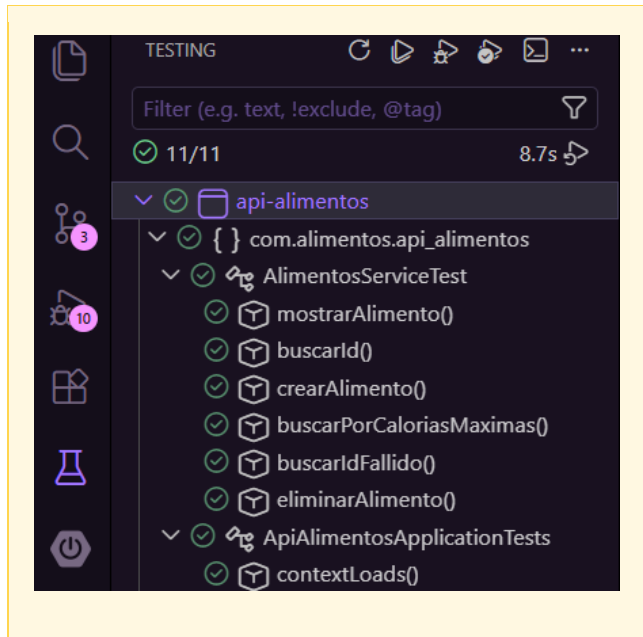
En estos métodos de prueba, se implementa el patrón AAA (Arrange, Act, Assert). Se utiliza `when(...).thenReturn(...)` para indicarle al mock del repositorio qué debe responder cuando el servicio lo

invoque. Tras ejecutar la acción, se utilizan aserciones (assertNotNull, assertEquals) para validar que los datos devueltos coincidan exactamente con lo esperado. Finalmente, la instrucción verify(...) asegura que el repositorio fue llamado exactamente la cantidad de veces esperadas.

```
82  @Test
83  @DisplayName("Buscar receta fallido - id no existe")
84  void buscarIdFallido() {
85      when(recetaRepository.findById(id: 999L)).thenReturn(Optional.empty());
86
87      assertThrows(NoSuchElementException.class, () -> {
88          recetaService.buscarId(id: 999L);
89      });
90
91      verify(recetaRepository, times(1)).findById(id: 999L);
92  }
93
94  @Test
95  @DisplayName("Eliminar una receta")
96  void eliminarReceta() {
97      doNothing().when(recetaRepository).deleteById(id: 1L);
98
99      recetaService.eliminarReceta(id: 1L);
100
101      verify(recetaRepository, times(1)).deleteById(id: 1L);
102  }
103 }
104
```

El test buscarIdFallido valida el manejo de errores. Simula que el repositorio devuelve un resultado vacío y utiliza assertThrows para confirmar que el servicio reacciona lanzando la excepción esperada (NoSuchElementException). El test eliminarReceta, al ser un método que no retorna valores (void), utiliza doNothing() para el mock y verifica con verify que la instrucción de borrado en el repositorio fue gatillada correctamente mediante el ID solicitado.

IV. DOCUMENTACIÓN DEL 60% DE COBERTURA



- ✓ nutricionista 1.3s
 - ✓ com.servicio.nutricionista 1.3s
 - ✓ NutricionistaApplicationTests 14ms
 - ✓ contextLoads() 14ms
 - ✓ NutricionistaServiceTest 1.3s
 - ✓ findAll_ReturnsListOfNutricionistaDTOs() Debería retornar una lista de nutricionistas: 6.0ms
 - ✓ findById_ExistingId_ReturnsNutricionistaDTO() Debería retornar un nutricionista cuando el ID existe: 4.0ms
 - ✓ findById_NonExistingId_ThrowsResourceNotFoundException() Debería lanzar ResourceNotFoundException cuando el ID no existe al buscar: 5.0ms
 - ✓ save_ValidDto_ReturnsSavedNutricionistaDTO() Debería guardar un nutricionista y retornar el DTO guardado: 9.0ms
 - ✓ update_ExistingIdAndValidDto_ReturnsUpdatedNutricionistaDTO() Debería actualizar un nutricionista cuando el ID existe: 5.0ms
 - ✓ update_NonExistingId_ThrowsResourceNotFoundException() Debería lanzar ResourceNotFoundException cuando el ID no existe al actualizar: 1.2s
 - ✓ delete_ExistingId_DeletesNutricionista() Debería eliminar un nutricionista cuando el ID existe: 9.0ms
 - ✓ delete_NonExistingId_ThrowsResourceNotFoundException() Debería lanzar ResourceNotFoundException cuando el ID no existe al eliminar: 5.0ms

- ✓ paciente 2.1s
 - ✓ com.servicio.paciente 2.1s
 - ✓ PacienteApplicationTests 758ms
 - ✓ contextLoads() 758ms
 - ✓ PacienteServiceTest 1.4s
 - ✓ testFindAll() Debería retornar una lista con todos los pacientes: 166ms
 - ✓ testSearchByNombreApellido_Ambos() Debería buscar por nombre y apellido: 8.0ms
 - ✓ testSearchByNombreApellido_SoloNombre() Debería buscar solo por nombre: 10ms
 - ✓ testSearchByNombreApellido_SoloApellido() Debería buscar solo por apellido: 12ms
 - ✓ testSearchByNombreApellido_Ninguno() Debería retornar lista vacía si no se envía nombre ni apellido: 1.1s
 - ✓ testFindById_Success() Debería retornar un paciente por su ID: 7.0ms
 - ✓ testFindById_NotFound() Debería lanzar excepción si no encuentra el paciente por ID: 8.0ms
 - ✓ testSave() Debería guardar un paciente correctamente: 26ms
 - ✓ testUpdate_Success() Debería actualizar un paciente correctamente: 11ms
 - ✓ testUpdate_NotFound() Debería lanzar excepción al actualizar un paciente inexistente: 18ms
 - ✓ testDelete_Success() Debería eliminar un paciente correctamente: 41ms
 - ✓ testDelete_NotFound() Debería lanzar excepción al intentar eliminar un paciente inexistente: 15ms

```
api-mediciones 1.3s
├── com.mediciones.api_mediciones 1.3s
│   └── ApiMedicionesApplicationTests 559ms
│       └── contextLoads() 559ms
└── MedicionServiceImplTest 762ms
    ├── createMedicion_Success() 4.0ms
    ├── getMedicionById_Success() 4.0ms
    ├── getMedicionById_NotFound() 6.0ms
    ├── getMedicionesByPacienteId_Success() 18ms
    ├── updateMedicion_Success() 10ms
    ├── updateMedicion_NotFound() 697ms
    ├── deleteMedicion_Success() 17ms
    └── deleteMedicion_NotFound() 6.0ms
```

```
api-metas 985ms
├── com.metast.api_metas 348ms
│   └── ApiMetasApplicationTests 348ms
│       └── contextLoads() 348ms
└── com.metast.api_metas.service.impl 637ms
    ├── MetaServiceImplTest 530ms
    │   ├── createMeta_Success() 31ms
    │   ├── getMetaById_Success() 4.0ms
    │   ├── getMetaById_NotFound() 5.0ms
    │   ├── getMetasByPacienteId_Success() 459ms
    │   ├── updateMeta_Success() 3.0ms
    │   ├── updateMeta_NotFound() 5.0ms
    │   ├── deleteMeta_Success() 12ms
    │   └── deleteMeta_NotFound() 11ms
    └── ProgresoMetaServiceImplTest 107ms
```

V. RESULTADO OBTENIDO AL TESTEAR

Al ejecutar las pruebas unitarias, JUnit genera informes detallados de la ejecución:

- Confirmación de éxito en color verde All Tests Passed.
- Verificación de tiempos de ejecución de las pruebas unitarias individuales.
- Ausencia de errores de regresión en las operaciones CRUD y validaciones complejas de negocio.


```

%TESTC 7 v2
%STSTREE2,com.alimentos.api_alimentos.AlimentosServiceTest,true,6,false,1,AlimentosServiceTest,,[engine:junit-jupiter]/[class:com.alimentos.api_alimentos.AlimentosServiceTest]
%STSTREE3,buscarId(com.alimentos.api_alimentos.AlimentosServiceTest),false,1,false,2,Buscar alimento por id,,[engine:junit-jupiter]/[class:com.alimentos.api_alimentos.AlimentosServiceTest]/[method:buscarId()]
%STSTREE4,buscarPorCaloriasMaximas(com.alimentos.api_alimentos.AlimentosServiceTest),false,1,false,2,Buscar alimentos por calorias mximas,,[engine:junit-jupiter]/[class:com.alimentos.api_alimentos.AlimentosServiceTest]/[method:buscarPorCaloriasMaximas()]
%STSTREE5,buscarIdFallido(com.alimentos.api_alimentos.AlimentosServiceTest),false,1,false,2,Buscar alimento fallido - id no existe,,[engine:junit-jupiter]/[class:com.alimentos.api_alimentos.AlimentosServiceTest]/[method:buscarIdFallido()]
%STSTREE6,mostrarAlimento(com.alimentos.api_alimentos.AlimentosServiceTest),false,1,false,2,Mostrar todos los alimentos,,[engine:junit-jupiter]/[class:com.alimentos.api_alimentos.AlimentosServiceTest]/[method:mostrarAlimento()]
%STSTREE7,eliminarAlimento(com.alimentos.api_alimentos.AlimentosServiceTest),false,1,false,2,Eliminar un alimento,,[engine:junit-jupiter]/[class:com.alimentos.api_alimentos.AlimentosServiceTest]/[method:eliminarAlimento()]
%STSTREE8,crearAlimento(com.alimentos.api_alimentos.AlimentosServiceTest),false,1,false,2,Crear un alimento,,[engine:junit-jupiter]/[class:com.alimentos.api_alimentos.AlimentosServiceTest]/[method:crearAlimento()]
%STSTREE9,com.alimentos.api_alimentos.ApiAlimentosApplicationTests,true,1,false,1,ApiAlimentosApplicationTests,,[engine:junit-jupiter]/[class:com.alimentos.api_alimentos.ApiAlimentosApplicationTests]
%STSTREE10,contextLoads(com.alimentos.api_alimentos.ApiAlimentosApplicationTests),false,1,false,9,contextLoads(),,[engine:junit-jupiter]/[class:com.alimentos.api_alimentos.ApiAlimentosApplicationTests]/[method:contextLoads()]
%TESTS 3,buscarId(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTE 3,buscarId(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTS 4,buscarPorCaloriasMaximas(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTE 4,buscarPorCaloriasMaximas(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTS 5,buscarIdFallido(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTE 5,buscarIdFallido(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTS 6,mostrarAlimento(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTE 6,mostrarAlimento(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTS 7,eliminarAlimento(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTE 7,eliminarAlimento(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTS 8,crearAlimento(com.alimentos.api_alimentos.AlimentosServiceTest)
%TESTE 8,crearAlimento(com.alimentos.api_alimentos.AlimentosServiceTest)

```

Test Runner for Java

- AlimentosServiceTest \$(symbol-namespace) com.alimentos.api_alimentos ...
- ApiAlimentosApplicationTests \$(symbol-namespace) com.alimentos.api_alimentos ...
- buscarId() \$(symbol-class) AlimentosServiceTest + \$(symbol-namespace) com...
- buscarIdFallido() \$(symbol-class) AlimentosServiceTest + \$(symbol-namespace) com...
- buscarPorCaloriasMaximas() \$(symbol-class) AlimentosServiceTest + \$(symbol-namespace) com...
- crearAlimento() \$(symbol-class) AlimentosServiceTest + \$(symbol-namespace) com...
- eliminarAlimento() \$(symbol-class) AlimentosServiceTest + \$(symbol-namespace) com...
- mostrarAlimento() \$(symbol-class) AlimentosServiceTest + \$(symbol-namespace) com...
- contextLoads() \$(symbol-class) ApiAlimentosApplicationTests + \$(symbol-namespace) com...

9 older results

```

%STSTREE2,com.recetas.api_recetas.ApiRecetasApplicationTests,true,1,false,1,ApiRecetasApplicationTests,,[engine:junit-jupiter]/[class:com.recetas.api_recetas.ApiRecetasApplicationTests]
%STSTREE3,contextLoads(com.recetas.api_recetas.ApiRecetasApplicationTests),false,1,false,2,contextLoads(),,[engine:junit-jupiter]/[class:com.recetas.api_recetas.ApiRecetasApplicationTests]/[method:contextLoads()]
%STSTREE4,com.recetas.api_recetas.RecetaServiceTest,true,5,false,1,RecetaServiceTest,,[engine:junit-jupiter]/[class:com.recetas.api_recetas.RecetaServiceTest]
%STSTREE5,eliminarReceta(com.recetas.api_recetas.RecetaServiceTest),false,1,false,4,Eliminar una receta,,[engine:junit-jupiter]/[class:com.recetas.api_recetas.RecetaServiceTest]/[method:eliminarReceta()]
%STSTREE6,buscarId(com.recetas.api_recetas.RecetaServiceTest),false,1,false,4,Buscar receta por id,,[engine:junit-jupiter]/[class:com.recetas.api_recetas.RecetaServiceTest]/[method:buscarId()]
%STSTREE7,buscarIdFallido(com.recetas.api_recetas.RecetaServiceTest),false,1,false,4,Buscar receta fallido - id no existe,,[engine:junit-jupiter]/[class:com.recetas.api_recetas.RecetaServiceTest]/[method:buscarIdFallido()]
%STSTREE8,crearReceta(com.recetas.api_recetas.RecetaServiceTest),false,1,false,4,Crear una receta,,[engine:junit-jupiter]/[class:com.recetas.api_recetas.RecetaServiceTest]/[method:crearReceta()]
%STSTREE9,mostrarRecetas(com.recetas.api_recetas.RecetaServiceTest),false,1,false,4,Mostrar todas las recetas,,[engine:junit-jupiter]/[class:com.recetas.api_recetas.RecetaServiceTest]/[method:mostrarRecetas()]
%TESTS 3,contextLoads(com.recetas.api_recetas.ApiRecetasApplicationTests)
%TESTE 3,contextLoads(com.recetas.api_recetas.ApiRecetasApplicationTests)
%TESTS 5,eliminarReceta(com.recetas.api_recetas.RecetaServiceTest)
%TESTE 5,eliminarReceta(com.recetas.api_recetas.RecetaServiceTest)
%TESTS 6,buscarId(com.recetas.api_recetas.RecetaServiceTest)
%TESTE 6,buscarId(com.recetas.api_recetas.RecetaServiceTest)
%TESTS 7,buscarIdFallido(com.recetas.api_recetas.RecetaServiceTest)
%TESTE 7,buscarIdFallido(com.recetas.api_recetas.RecetaServiceTest)
%TESTS 8,crearReceta(com.recetas.api_recetas.RecetaServiceTest)
%TESTE 8,crearReceta(com.recetas.api_recetas.RecetaServiceTest)
%TESTS 9,mostrarRecetas(com.recetas.api_recetas.RecetaServiceTest)
%TESTE 9,mostrarRecetas(com.recetas.api_recetas.RecetaServiceTest)
%RUNTIME16424

```

Test Runner for Java

- ApiRecetasApplicationTests \$(symbol-namespace) com.recetas.api_recetas ...
- RecetaServiceTest \$(symbol-namespace) com.recetas.api_recetas + \$(project)...
- contextLoads() \$(symbol-class) ApiRecetasApplicationTests + \$(symbol-namespace) com...
- buscarId() \$(symbol-class) RecetaServiceTest + \$(symbol-namespace) com...
- buscarIdFallido() \$(symbol-class) RecetaServiceTest + \$(symbol-namespace) com...
- crearReceta() \$(symbol-class) RecetaServiceTest + \$(symbol-namespace) com...
- eliminarReceta() \$(symbol-class) RecetaServiceTest + \$(symbol-namespace) com...
- mostrarRecetas() \$(symbol-class) RecetaServiceTest + \$(symbol-namespace) com...

10 older results

```

%TSTREE12,contextLoads(com.servicio.nutricionista.NutricionistaApplicationTests),false,1,false,11,contextLoads(),,[engine:j
unit-jupiter]/[class:com.servicio.nutricionista.NutricionistaApplicationTests]/[method:contextLoads()]
%TESTS 3,update_NonExistingId_ThrowsResourceNotFoundException(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTE 3,update_NonExistingId_ThrowsResourceNotFoundException(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTS 4,save_ValidDto_ReturnsSavedNutricionistaDTO(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTE 4,save_ValidDto_ReturnsSavedNutricionistaDTO(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTS 5,findAll_ReturnsListOfNutricionistaDTOS(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTE 5,findAll_ReturnsListOfNutricionistaDTOS(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTS 6,update_ExistingIdAndValidDto_ReturnsUpdatedNutricionistaDTO(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTE 6,update_ExistingIdAndValidDto_ReturnsUpdatedNutricionistaDTO(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTS 7,delete_ExistingId_DeletesNutricionista(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTE 7,delete_ExistingId_DeletesNutricionista(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTS 8,delete_NonExistingId_ThrowsResourceNotFoundException(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTE 8,delete_NonExistingId_ThrowsResourceNotFoundException(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTS 9,findById_NonExistingId_ThrowsResourceNotFoundException(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTE 9,findById_NonExistingId_ThrowsResourceNotFoundException(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTS 10,findById_ExistingId_ReturnsNutricionistaDTO(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTE 10,findById_ExistingId_ReturnsNutricionistaDTO(com.servicio.nutricionista.NutricionistaServiceTest)

%TESTS 12,contextLoads(com.servicio.nutricionista.NutricionistaApplicationTests)

%TESTE 12,contextLoads(com.servicio.nutricionista.NutricionistaApplicationTests)

%RUNTIME14196

```

Test Runner for Java

- ✓ NutricionistaServiceTest \$(symbol-namespace) com.servicio.nutricionista + 1...
- ✓ NutricionistaApplicationTests \$(symbol-namespace) com.servicio.nutricionista + 1...
- ✓ delete_ExistingId_DeletesNutricionista() \$(symbol-class) NutricionistaServ...
- ✓ delete_NonExistingId_ThrowsResourceNotFoundException() \$(symbol-cl...
- ✓ findAll_ReturnsListOfNutricionistaDTOS() \$(symbol-class) NutricionistaSer...
- ✓ findById_ExistingId_ReturnsNutricionistaDTO() \$(symbol-class) Nutricioni...
- ✓ findById_NonExistingId_ThrowsResourceNotFoundException() \$(symbo...
- ✓ save_ValidDto_ReturnsSavedNutricionistaDTO() \$(symbol-class) Nutricion...
- ✓ update_ExistingIdAndValidDto_ReturnsUpdatedNutricionistaDTO() \$(s...
- ✓ update_NonExistingId_ThrowsResourceNotFoundException() \$(symbol-na...
- ✓ contextLoads() \$(symbol-class) NutricionistaApplicationTests + \$(symbol-na...

> 11 older results

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS
%TESTS 7,testDelete_Success(com.servicio.paciente.PacienteServiceTest)

%TESTE 7,testDelete_Success(com.servicio.paciente.PacienteServiceTest)

%TESTS 8,testSave(com.servicio.paciente.PacienteServiceTest)

%TESTE 8,testSave(com.servicio.paciente.PacienteServiceTest)

%TESTS 9,testSearchByNombreApellido_SoloNombre(com.servicio.paciente.PacienteServiceTest)

%TESTE 9,testSearchByNombreApellido_SoloNombre(com.servicio.paciente.PacienteServiceTest)

%TESTS 10,testSearchByNombreApellido_SoloApellido(com.servicio.paciente.PacienteServiceTest)

%TESTE 10,testSearchByNombreApellido_SoloApellido(com.servicio.paciente.PacienteServiceTest)

%TESTS 11,testDelete_NotFound(com.servicio.paciente.PacienteServiceTest)

%TESTE 11,testDelete_NotFound(com.servicio.paciente.PacienteServiceTest)

%TESTS 12,testFindById_Success(com.servicio.paciente.PacienteServiceTest)

%TESTE 12,testFindById_Success(com.servicio.paciente.PacienteServiceTest)

%TESTS 13,testUpdate_NotFound(com.servicio.paciente.PacienteServiceTest)

%TESTE 13,testUpdate_NotFound(com.servicio.paciente.PacienteServiceTest)

%TESTS 14,testSearchByNombreApellido_Ambos(com.servicio.paciente.PacienteServiceTest)

%TESTE 14,testSearchByNombreApellido_Ambos(com.servicio.paciente.PacienteServiceTest)

%TESTS 15,testFindById_NotFound(com.servicio.paciente.PacienteServiceTest)

%TESTE 15,testFindById_NotFound(com.servicio.paciente.PacienteServiceTest)

%TESTS 16,testUpdate_Success(com.servicio.paciente.PacienteServiceTest)

%TESTE 16,testUpdate_Success(com.servicio.paciente.PacienteServiceTest)

%RUNTIME14978

```

Test Runner for Java

- ✓ PacienteApplicationTests \$(symbol-namespace) com.servicio.paciente + \$(project)...
- ✓ PacienteServiceTest \$(symbol-namespace) com.servicio.paciente + \$(project)...
- ✓ contextLoads() \$(symbol-class) PacienteApplicationTests + \$(symbol-namesp...
- ✓ testDelete_NotFound() \$(symbol-class) PacienteServiceTest + \$(symbol-nam...
- ✓ testDelete_Success() \$(symbol-class) PacienteServiceTest + \$(symbol-names...
- ✓ testFindAll() \$(symbol-class) PacienteServiceTest + \$(symbol-namespace) co...
- ✓ testFindById_NotFound() \$(symbol-class) PacienteServiceTest + \$(symbol-na...
- ✓ testFindById_Success() \$(symbol-class) PacienteServiceTest + \$(symbol-nam...
- ✓ testSave() \$(symbol-class) PacienteServiceTest + \$(symbol-namespace) com...
- ✓ testSearchByNombreApellido_Ambos() \$(symbol-class) PacienteServiceTe...
- ✓ testSearchByNombreApellido_Ninguno() \$(symbol-class) PacienteService...
- ✓ testSearchByNombreApellido_SoloApellido() \$(symbol-class) PacienteSer...
- ✓ testSearchByNombreApellido_SoloNombre() \$(symbol-class) PacienteSer...
- ✓ testUpdate_NotFound() \$(symbol-class) PacienteServiceTest + \$(symbol-na...
- ✓ testUpdate_Success() \$(symbol-class) PacienteServiceTest + \$(symbol-name...

> 12 older results

```
Test Runner for %
  ✓ ApiMetaApplicationTests % (symbol=namespace) com.net.api_meta.application
  ✓ MetaServiceImplTest % (symbol=namespace) com.net.api_meta.service
  ✓ Progresso_MetaNotFound % (symbol=namespace) com.net.api_meta.application
  ✓ context.oas3 % (symbol=class) ApiMetaApplicationTests % (symbol=
  ✓ createMeta_Success % (symbol=class) MetaServiceServiceImplTest % (symbol=
  ✓ deleteMeta_NotFound % (symbol=class) MetaServiceServiceImplTest % (symbol=
  ✓ deleteMeta_Success % (symbol=class) MetaServiceServiceImplTest % (symbol=
  ✓ getMetaById_NotFound % (symbol=class) MetaServiceServiceImplTest % (symbol=
  ✓ getMetaById_Success % (symbol=class) MetaServiceServiceImplTest % (symbol=
  ✓ getMetaByPatientId_Success % (symbol=class) MetaServiceServiceImplTest % (
  ✓ updateMeta_NotFound % (symbol=class) MetaServiceServiceImplTest % (symbol=
  ✓ updateMeta_Success % (symbol=class) MetaServiceServiceImplTest % (symbol=
  ✓ addProgresso_MetaNotFound % (symbol=class) ProgressoMetaServiceImplTest %
  ✓ addProgresso_Success % (symbol=class) ProgressoMetaServiceImplTest % (
  ✓ deleteProgresso_NotFound % (symbol=class) ProgressoMetaServiceImplTest %
  ✓ deleteProgresso_Success % (symbol=class) ProgressoMetaServiceImplTest %
  ✓ getProgressoByMetaId_MetaNotFound % (symbol=class) ProgressoMetaSe
  ✓ getProgressoByMetaId_Success % (symbol=class) ProgressoMetaSe
```

```

test Runner for Java
  @ ApiMeta(ApplicationTests $symbol-namepace) com.meta.api.meta; $!
  @ MetaServiceImplTest $symbol-namepace) com.meta.api.meta.service.im
  @ ProgressoMetaServiceImplTest $symbol-namepace) com.meta.api.meta.s
  context(coadj) $symbol-class) ApiMeta(ApplicationTests $symbol-name
  @ createMeta_Success $symbol-class) MetaServiceImplTest $symbol-na
  @ deleteMeta_NotFound $symbol-class) MetaServiceImplTest $symbol-na
  @ deleteMeta_Success $symbol-class) MetaServiceImplTest $symbol-na
  @ getMetaById_NotFound $symbol-class) MetaServiceImplTest $symbol-
  @ getMetaById_Success $symbol-class) MetaServiceImplTest $symbol-na
  @ getMetaByPatientId_Success $symbol-class) MetaServiceImplTest $sy
  @ updateMeta_NotFound $symbol-class) MetaServiceImplTest $symbol-na
  @ updateMeta_Success $symbol-class) MetaServiceImplTest $symbol-na
  @ addProgresso_MetaNotFound $symbol-class) ProgressoMetaServiceImpl
  @ addProgresso_Success $symbol-class) ProgressoMetaServiceImplTest $
  @ deleteProgresso_NotFound $symbol-class) ProgressoMetaServiceImplTest
  @ deleteProgresso_Success $symbol-class) ProgressoMetaServiceImplTest $
  @ getProgressoByMetaId_MetaNotFound $symbol-class) ProgressoMetaSe
  @ getProgressoByMetaId_Success $symbol-class) ProgressoMetaServiceIm...
3 order results

```